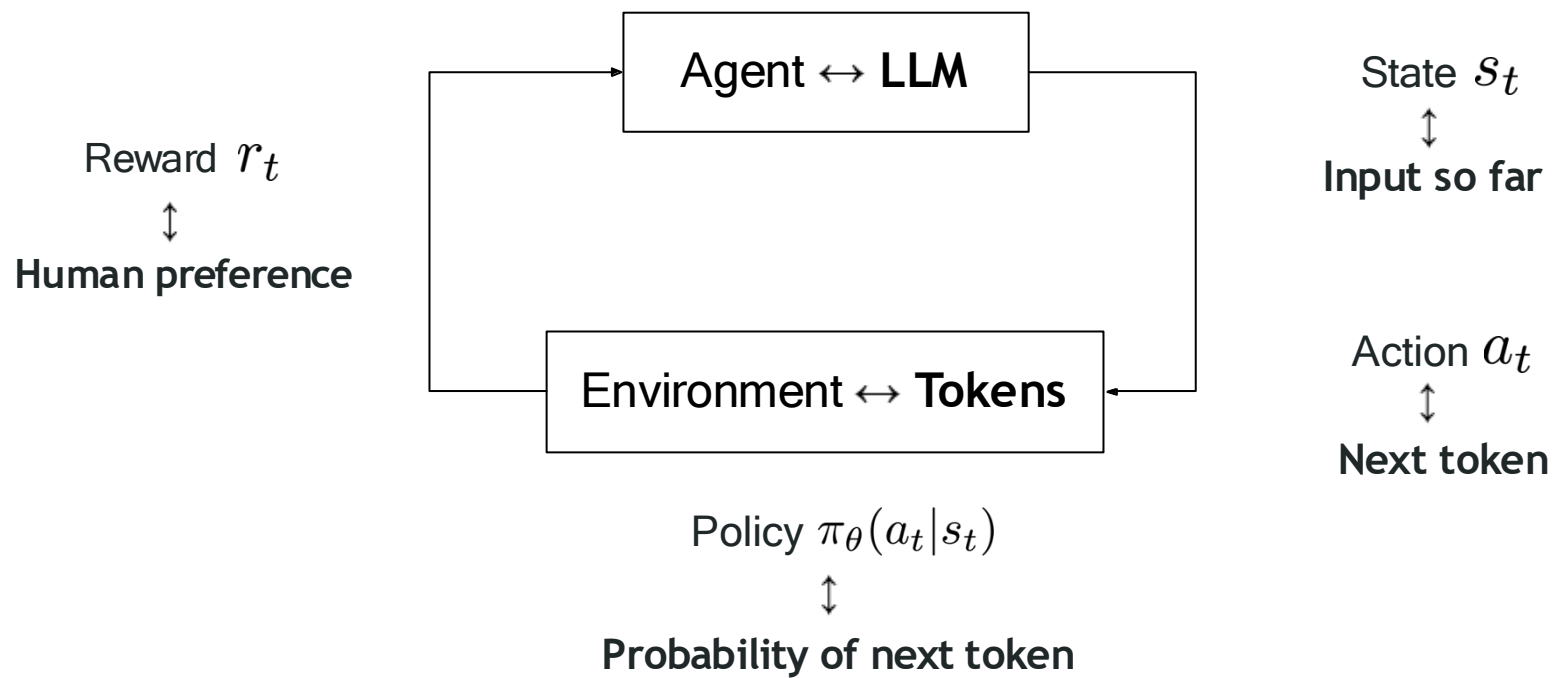


Deep Learning

Lecture 12

Recall



Last time PPO

- Many model weights; complex

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\overset{1}{\boxed{\pi_{\theta}(a_t | s_t)}} \overset{3}{\boxed{\hat{A}_t}}}{\underset{2}{\boxed{\pi_{\theta_{old}}(a_t | s_t)}} \underset{4}{\boxed{}}} - \beta \text{KL}[\overset{2}{\boxed{\pi_{\theta_{old}}(\cdot | s_t)}}, \overset{1}{\boxed{\pi_{\theta}(\cdot | s_t)}}] \right]$$

Can we learn in a supervised manner?

- **Try to simplify PPO by..**
 - Getting rid of the reward model
 - Getting rid of any on-policy stuff (rollouts, outer loops etc)
- **Attempts:**
 - Train the model with a control token , i.e., SFT on the pairs, prepend [GOOD] to chosen, [BAD] to not chosen.
 - Train the model on only preferred output
 - Train a reward model, get LM outputs, train on the preferred output
 - Train a reward model, get 1024 LM outputs, take the best one (Best-of-N)

Direct Preference Optimization (DPO)

- Take gradient steps on log-loss of good stuff
- Take negative gradient steps on bad stuff (appropriately weighted).

Direct Preference Optimization (DPO)

- DPO loss function

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\overbrace{\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)}}^{\text{Increase "good"}} - \overbrace{\beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)}}^{\text{Decrease "bad"}} \right) \right]$$

- Take gradient steps on log-loss of good stuff
- Take negative gradient steps on bad stuff (appropriately weighted).
- No need to train a separate reward model
- Operates directly on preference data
- Note, similarity to the Bradley-Terry formulation with special kind of reward:
$$r_{\theta}(x, y) = \beta \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$$

Let's derive DPO

- Start from PPO Objective

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} [r_{\phi}(x, y)] - \beta \mathbb{D}_{\text{KL}} [\pi_{\theta}(y | x) || \pi_{\text{ref}}(y | x)]$$

Let's derive DPO

- Start from PPO Objective
- Derive optimal policy

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp \left(\frac{1}{\beta} r^*(x, y) \right)$$

Let's derive DPO

- Start from PPO Objective
- Derive optimal policy
- Identify a “reward” term

$$r^*(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

Let's derive DPO

- Start from PPO Objective
- Derive optimal policy
- Identify a “reward” term
- Write Bradley-Terry formulation for this “reward”

$$p^*(y_w \succ y_l \mid x) = \frac{1}{1 + \exp \left(\beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} - \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} \right)}$$

Let's derive DPO

- Start from PPO Objective
- Derive optimal policy
- Identify a “reward” term
- Write Bradley-Terry formulation for this “reward”
- “Infer” DPO loss function

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Key takeaways

- Make a nonparametric assumption (links π_θ and R in closed form)
- Parametrize reward via the policy
- Optimize the reward using supervised losses (which in turn, optimizes the policy)
- Conceptually: This is MLE on the pairwise rewards, under nonparametric assumption + alternative parametrization

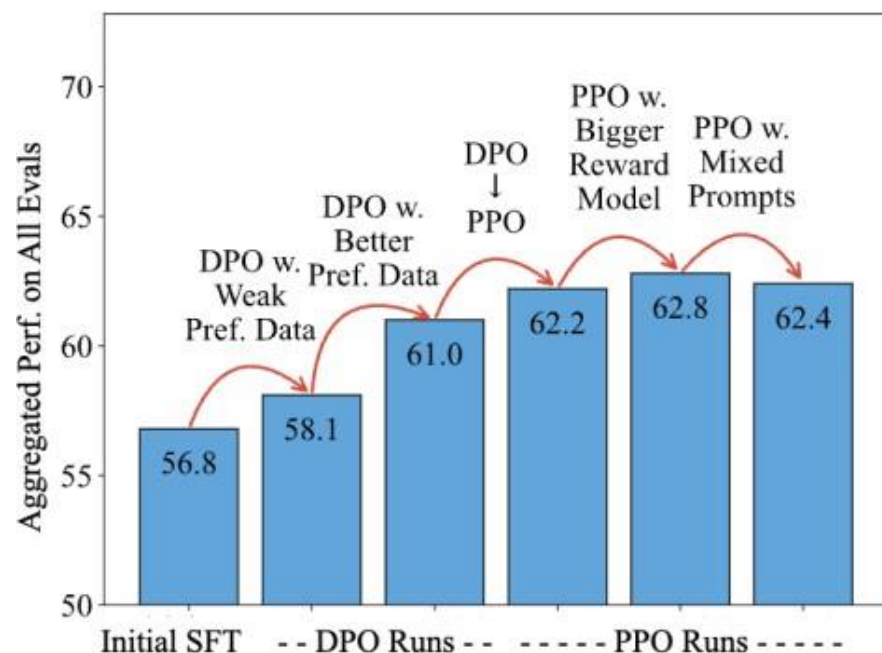
Performance

- No common absolute consensus. Varies from task to task and sensitive to implementation.

Method	Simulated Win-rate (%)	Human Win-rate (%)
GPT-4* [†]	79.0 ± 1.4	69.8 ± 1.6
ChatGPT* [†]	61.4 ± 1.7	52.9 ± 1.7
PPO	46.8 ± 1.8	55.1 ± 1.7
DPO	46.8 ± 1.7	-
Best-of-1024	45.0 ± 1.7	50.7 ± 1.8
Expert Iteration	41.9 ± 1.7	45.7 ± 1.7
SFT 52k	39.2 ± 1.7	40.7 ± 1.7
SFT 10k	36.7 ± 1.7	44.3 ± 1.7
Binary FeedME	36.6 ± 1.7	37.9 ± 1.7
Quark	35.6 ± 1.7	-
Binary Reward Conditioning	32.4 ± 1.6	-
Davinci001*	24.4 ± 1.5	32.5 ± 1.6
LLaMA 7B*	11.3 ± 1.1	6.5 ± 0.9

PPO or DPO?

- Mixing?*



- Different setup different results**

Algorithm	LR	$\gamma - \beta$ ratio	β	Epochs	Batch Size	Average Score
SFT Base	-	-	-	-	-	55.7
SimPO	5.00E-07	0.5	2	1	128	51.8
SimPO	5.00E-07	0.3	10	1	128	52.9
DPO	5.00E-07	-	0.1	3	32	55.2
PPO	1.00E-06	-	0.0325	1	64	54.5
PPO	1.00E-06	-	0.05	1	64	55.5
DPO-norm	1.00E-07	-	5	3	32	56.1
DPO-norm	5.00E-07	-	10	3	32	55.2
DPO-norm	5.00E-07	-	15	3	32	55.7
DPO-norm	5.00E-07	-	2	3	32	46.8
DPO-norm	5.00E-07	-	5	3	32	53.4
DPO-norm	5.00E-07	-	5	1	32	57.3

(*) Unpacking PPO and DPO; Ivsiuin et al.

(**) Tulu 3; Lambert et al.

DPO works

- Most 'top open-source' RLHF models are DPO'd

T ▲	Model	Average 📈 ▲	ARC ▲	HellaSwag ▲	MMLU ▲	TruthfulQA ▲	Winogrande ▲	GSM8K ▲
■	udkai/Turdus	74.66	73.38	88.56	64.52	67.11	86.66	67.7
■	fblgit/UNA-TheBeagle-7b-v1	73.87	73.04	88	63.48	69.85	82.16	66.72
■	argilla/distilabeled-Marcoro14-7B-slerp	73.63	70.73	87.47	65.22	65.1	82.08	71.19
■	mlabonne/NeuralMarcoro14-7B	73.57	71.42	87.59	64.84	65.64	81.22	70.74
◆	abideen/NexoNimbus-7B	73.5	70.82	87.86	64.69	62.43	84.85	70.36
■	Neuronovo/neuronovo-7B-v0.2	73.44	73.04	88.32	65.15	71.02	80.66	62.47
■	argilla/distilabeled-Marcoro14-7B-slerp-full	73.4	70.65	87.55	65.33	64.21	82	70.66
■	CultriX/MistralTrix-v1	73.39	72.27	88.33	65.24	70.73	80.98	62.77
■	ryandt/MusingCaterpillar	73.33	72.53	88.34	65.26	70.93	80.66	62.24
■	Neuronovo/neuronovo-7B-v0.3	73.29	72.7	88.26	65.1	71.35	80.9	61.41
■	CultriX/MistralTrixTest	73.17	72.53	88.4	65.22	70.77	81.37	60.73
◆	samir-fama/SamirGPT-v1	73.11	69.54	87.04	65.3	63.37	81.69	71.72
◆	SanjiWatsuki/Lelantos-DPO-7B	73.09	71.08	87.22	64	67.77	80.03	68.46

Handwritten notes in red:

- DPO (above udkai/Turdus)
- DPO (& UNA) (above fblgit/UNA-TheBeagle-7b-v1)
- DPO (above argilla/distilabeled-Marcoro14-7B-slerp)
- DPO (above mlabonne/NeuralMarcoro14-7B)
- Merge (of DPO models) (above abideen/NexoNimbus-7B)
- DPO (above Neuronovo/neuronovo-7B-v0.2)
- DPO (above argilla/distilabeled-Marcoro14-7B-slerp-full)
- DPO (above CultriX/MistralTrix-v1)
- DPO (above ryandt/MusingCaterpillar)
- DPO (above Neuronovo/neuronovo-7B-v0.3)
- No info but prob DPO, given Merge (incl. DPO) (above CultriX/MistralTrixTest)
- DPO (above samir-fama/SamirGPT-v1)
- DPO (above SanjiWatsuki/Lelantos-DPO-7B)

Variants

- Lots of variants (too many variants), but maybe two of note from the Tulu 3 paper
- SimPO(noref)

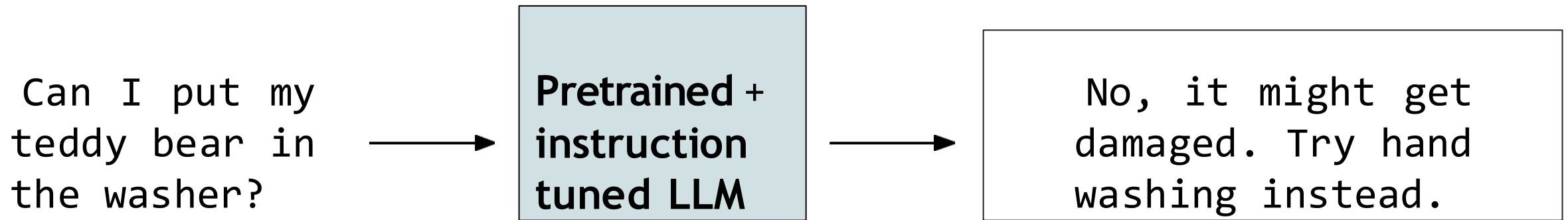
$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

$$\mathcal{L}_{\text{SimPO}}(\pi_{\theta}) = -\mathbb{E} \left[\log \sigma \left(\frac{\beta}{|y_w|} \log \pi_{\theta}(y_w | x) - \frac{\beta}{|y_l|} \log \pi_{\theta}(y_l | x) - \gamma \right) \right]$$

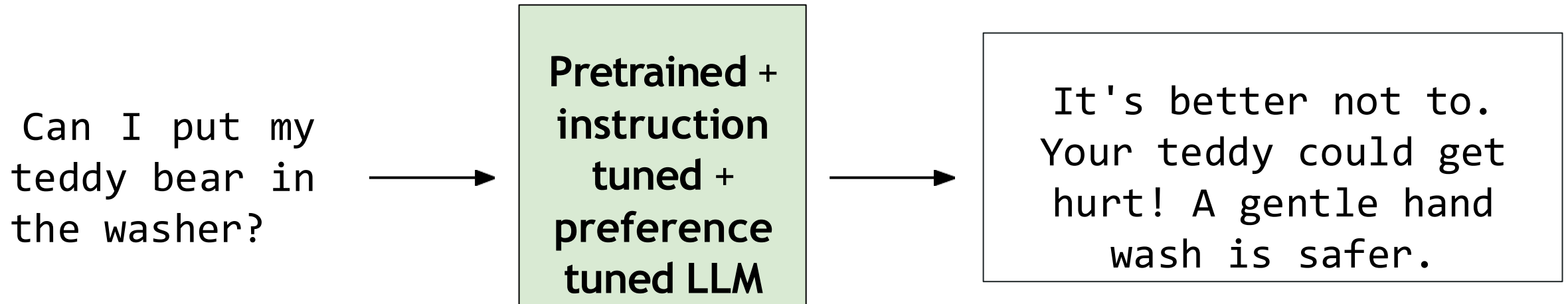
- Length normalized DPO

$$\max_{\pi_{\theta}} \mathbb{E}_{y_c, y_r \sim \mathcal{D}} \left[\log \sigma \left(\frac{\beta}{|y_c|} \log \frac{\pi_{\theta}(y_c | x)}{\pi_{\text{ref}}(y_c | x)} - \frac{\beta}{|y_r|} \log \frac{\pi_{\theta}(y_r | x)}{\pi_{\text{ref}}(y_r | x)} \right) \right].$$

Summary Preference tuning



Summary Preference tuning



Summary Preference tuning

RLHF	DPO
• Multi-stage training • Needs extra models: reward model, value model, base model	• Supervised learning • Base model is the only extra model needed

Do we need another RL algorithm?

- Why not PPO?
 - In practice, complicated implementation
 - Value model (memory hungry, involves additional tuning for training)
- Why not DPO?
 - Data not inherently pairwise (or in the form of Bradley-terry comparisons)
 - Offline (though could be made online by iterating)

Group Relative Policy Optimization (GRPO)

$$\mathcal{L}(\theta) = \boxed{\text{Maximize advantages}} + \boxed{\text{Don't deviate too much from old / base model}}$$

$$\boxed{\text{Advantage} \sim \text{Reward} - \text{Avg}(\text{reward of group})}$$



Big difference compared to PPO!

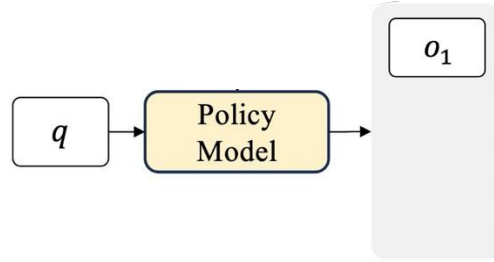
How GRPO works

GRPO

q

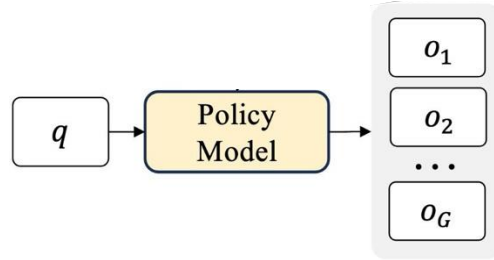
How GRPO works

GRPO



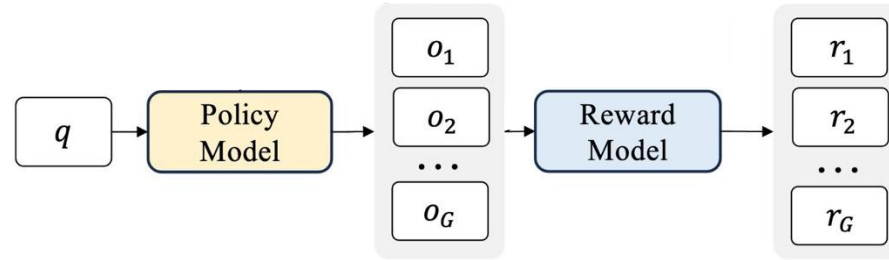
How GRPO works

GRPO



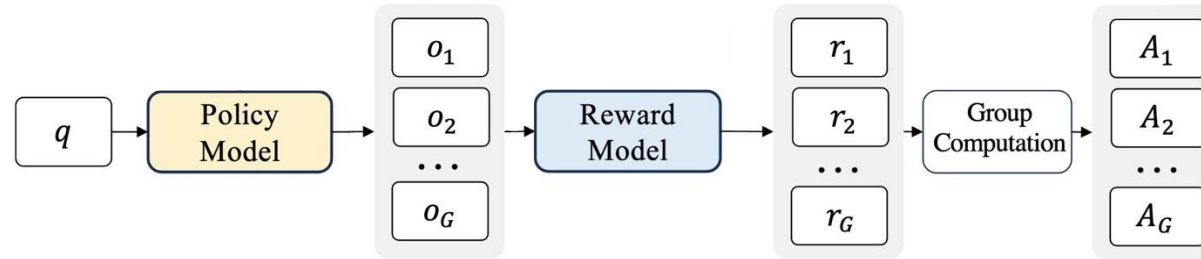
How GRPO works

GRPO



How GRPO works

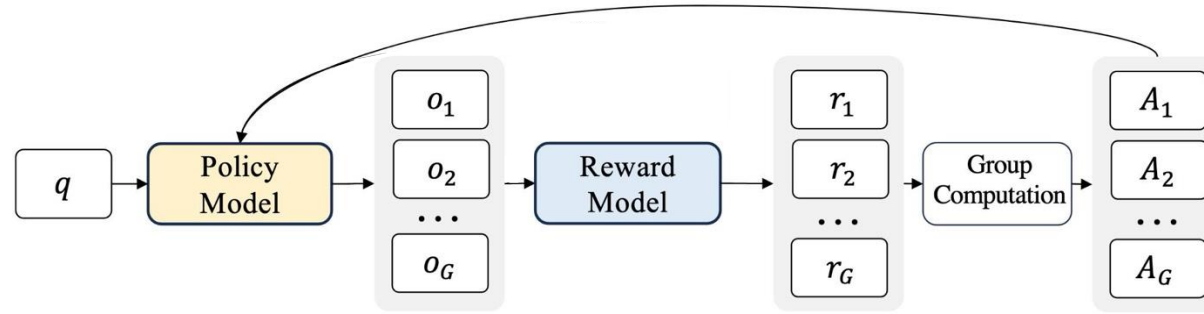
GRPO



$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

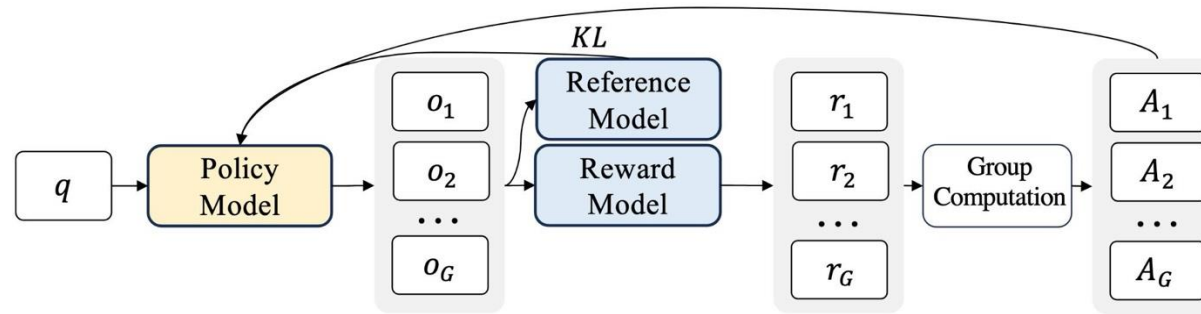
How GRPO works

GRPO



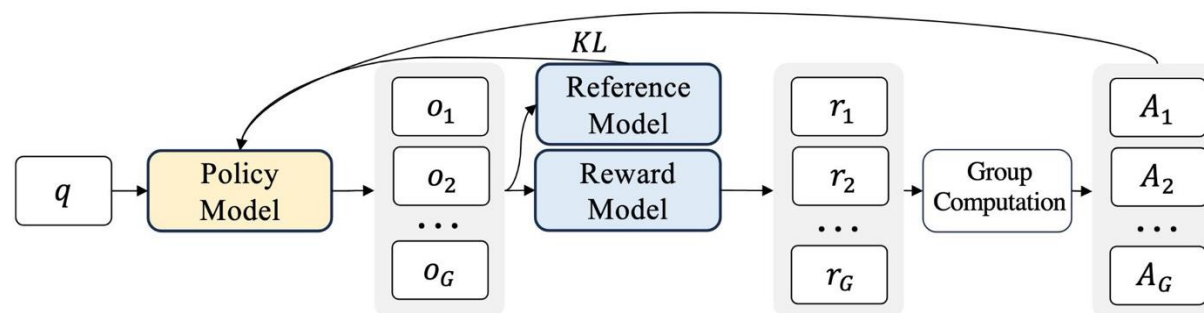
How GRPO works

GRPO



Comparison to PPO

GRPO

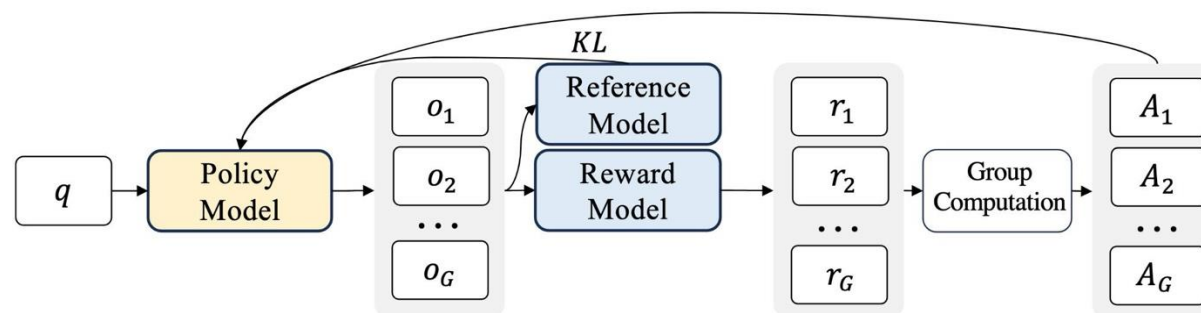


PPO

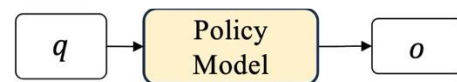


Comparison to PPO

GRPO

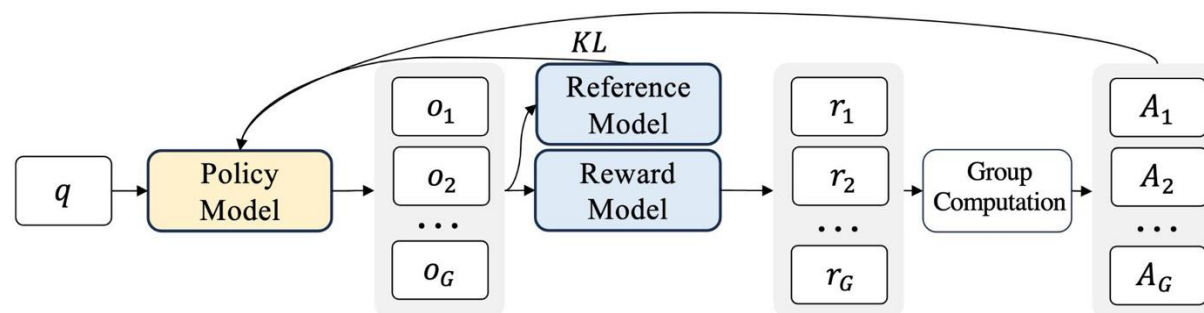


PPO

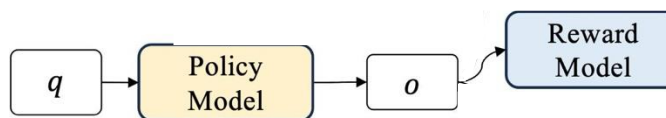


Comparison to PPO

GRPO

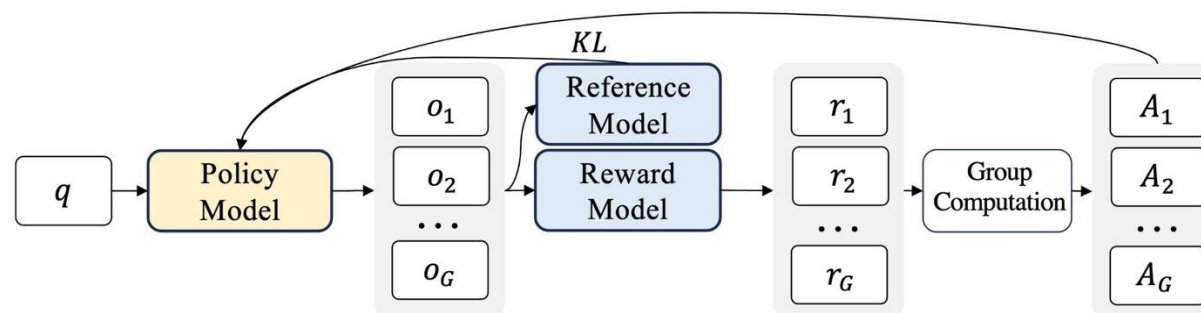


PPO

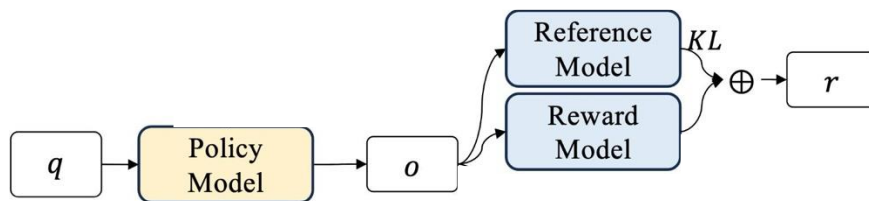


Comparison to PPO

GRPO

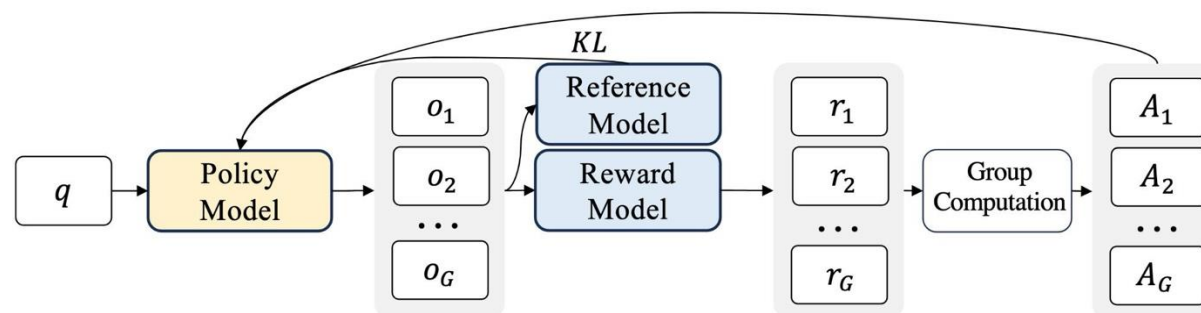


PPO

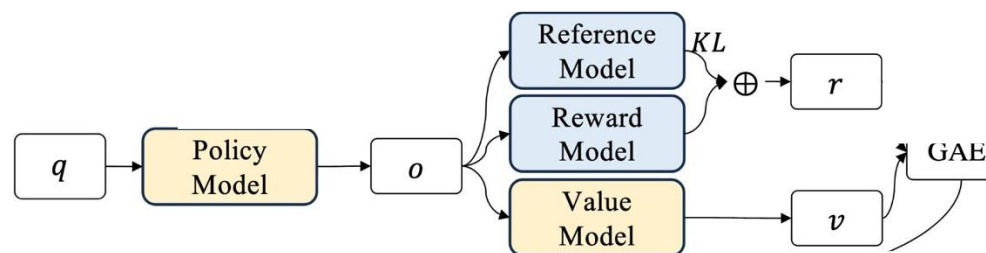


Comparison to PPO

GRPO

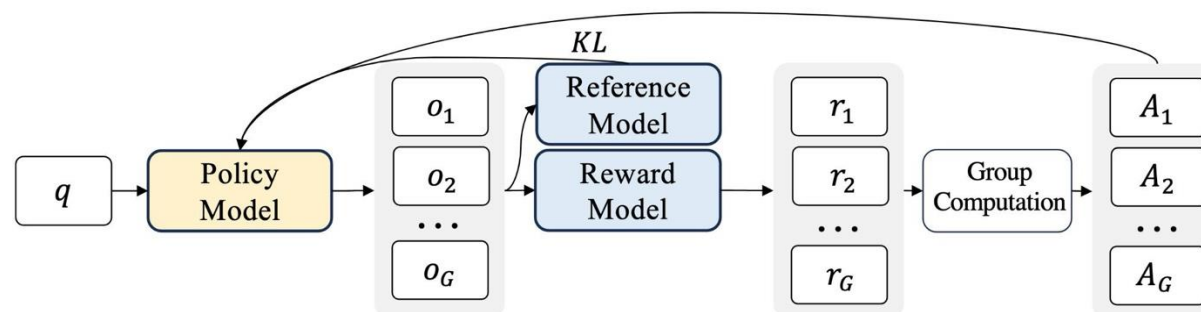


PPO

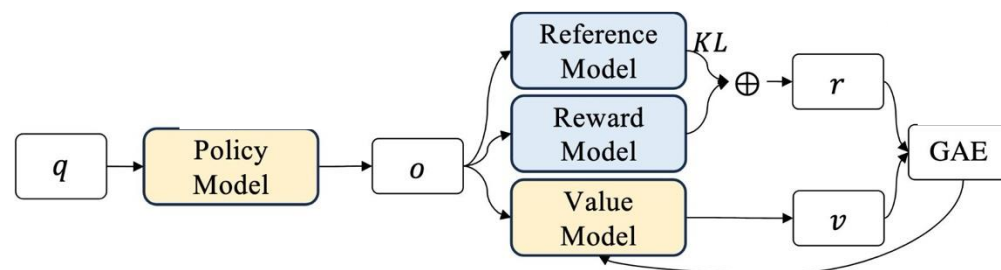


Comparison to PPO

GRPO

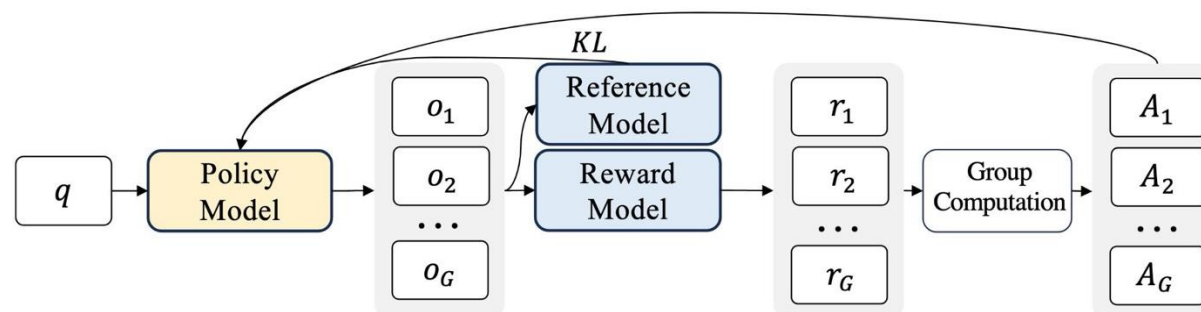


PPO

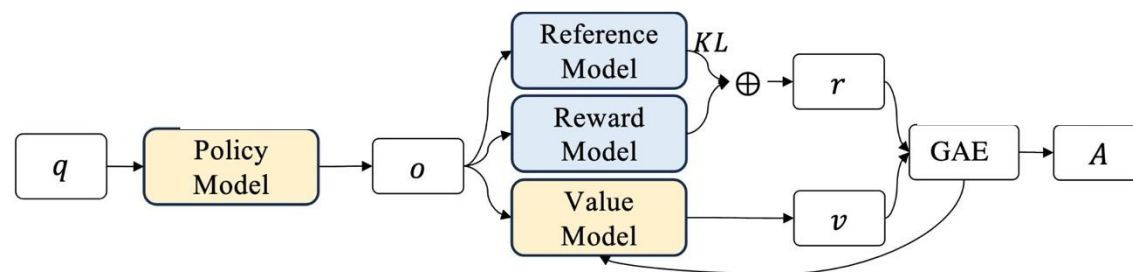


Comparison to PPO

GRPO

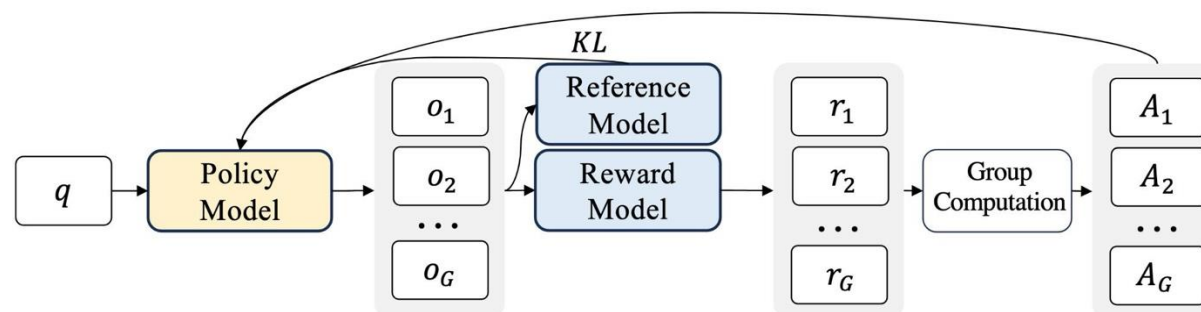


PPO

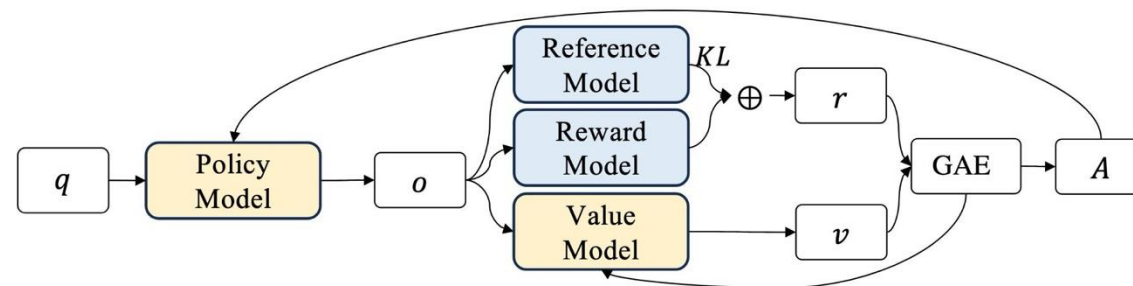


Comparison to PPO

GRPO

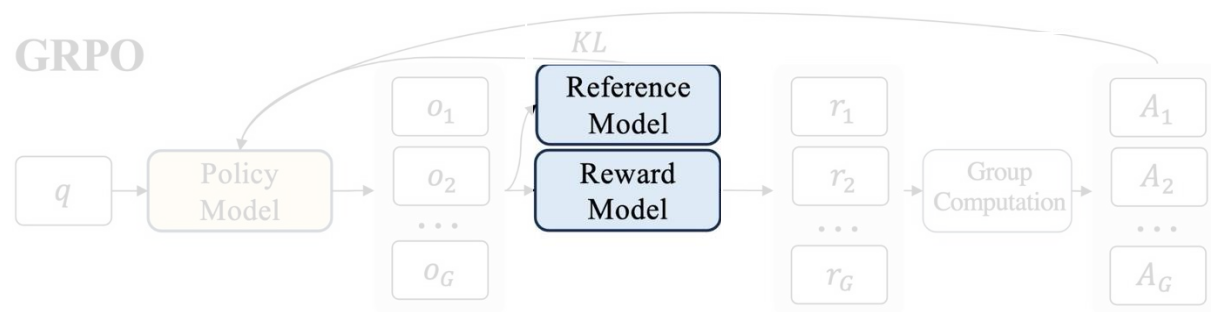


PPO

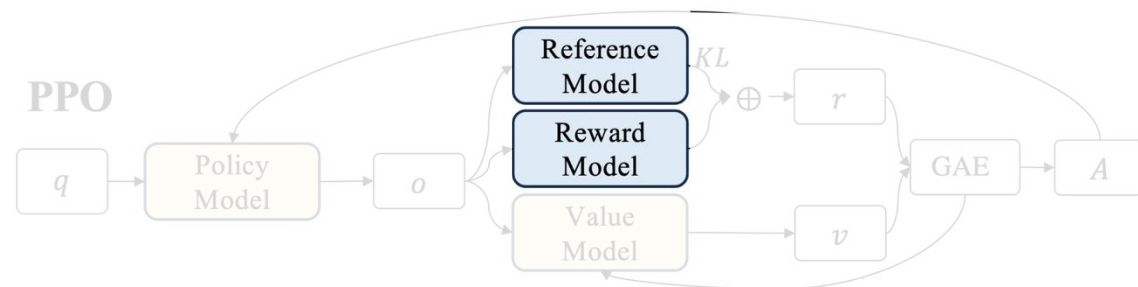


Comparison to PPO

GRPO



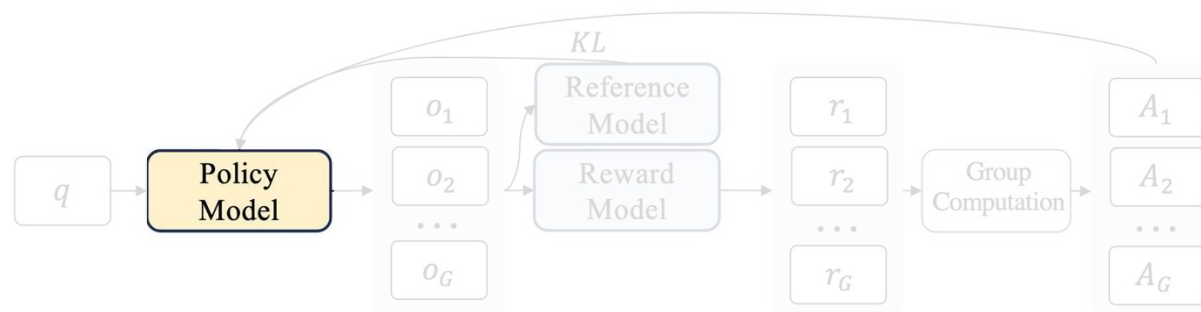
PPO



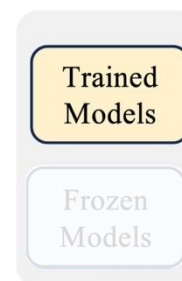
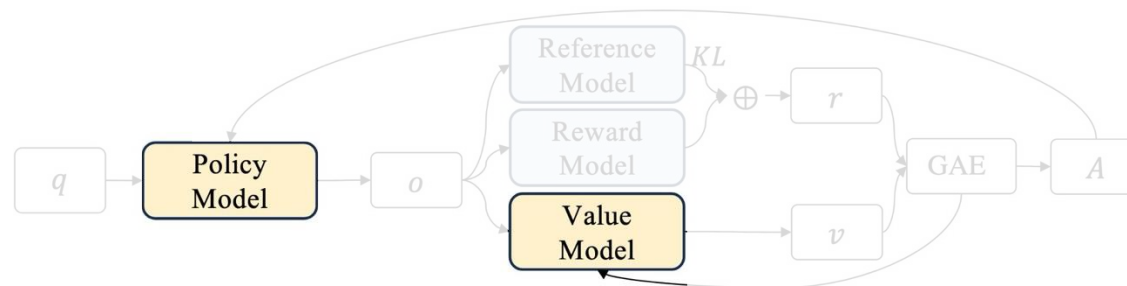
Frozen Models

Comparison to PPO

GRPO



PPO



Comparison to PPO

GRPO

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

PPO

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right]$$

Comparison to PPO

Similarities. Ratio

GRPO

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

PPO

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right]$$

Comparison to PPO

Similarities. Ratio, clipping

GRPO

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

PPO

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right]$$

Comparison to PPO

Differences. KL penalty

GRPO

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

PPO

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right]$$

Comparison to PPO

Differences. KL penalty, advantage estimation

GRPO

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

PPO

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right]$$

GRPO is very simple (thanks to lack of value function..)

- You can (and people do) write tiny GRPO implementations
 - Compute reward for each rollout
 - Mean/Var normalization per group
 - Compute KL term
 - Gradient updates on the loss

```
def compute_pg_loss(
    policy_model: Union[DeepSpeedEngine, PreTrainedModel],
    reference_model: Union[DeepSpeedEngine, PreTrainedModel],
    batch: Dict[str, torch.Tensor],
    total_response_len: int,
) -> Tuple[torch.Tensor, Dict[str, float]]:
    """
    Compute the policy gradient loss with KL penalty between policy and reference models.

    This function:
    1. Computes log probabilities for both policy and reference models
    2. Calculates KL divergence penalty between the models
    3. Computes policy gradient loss using advantages
    4. Combines the losses with KL coefficient

    Args:
        policy_model: The model being trained
        reference_model: The reference model for KL penalty calculation
        batch: Dictionary containing:
            - input_ids: Tensor of shape [batch_size, seq_len]
            - attention_mask: Tensor of shape [batch_size, seq_len]
            - labels: Tensor of shape [batch_size, seq_len] with -100 for ignored positions
            - advantages: Tensor of shape [batch_size, seq_len]

    Returns:
        Tuple containing:
        - loss: Combined policy gradient and KL penalty loss (scalar tensor)
        - metrics: Dictionary with detailed loss components:
            - policy_loss: Pure policy gradient loss
            - kl_penalty: KL divergence penalty
            - entropy: Policy entropy
    """
    input_ids = batch["input_ids"] # [batch_size, seq_len]
    attention_mask = batch["attention_mask"] # [batch_size, seq_len]
    labels = batch["labels"] # [batch_size, seq_len]
    advantages = batch["advantages"] # [batch_size, seq_len]

    model_inputs = {
        "input_ids": input_ids,
        "attention_mask": attention_mask,
        "labels": labels,
    }

    labels_mask = (labels[..., 1:] != -100).float() # [batch_size, seq_len-1]

    with torch.no_grad():
        ref_logps = compute_token_log_probs(
            reference_model, model_inputs, TEMPERATURE
        ) # [batch_size, seq_len-1]

    logps = compute_token_log_probs(policy_model, model_inputs, TEMPERATURE) # [batch_size, seq_len-1]

    kl_penalty = torch.exp(ref_logps - logps) - (ref_logps - logps) - 1 # [batch_size, seq_len-1]
    kl_penalty = kl_penalty * labels_mask # [batch_size, seq_len-1]

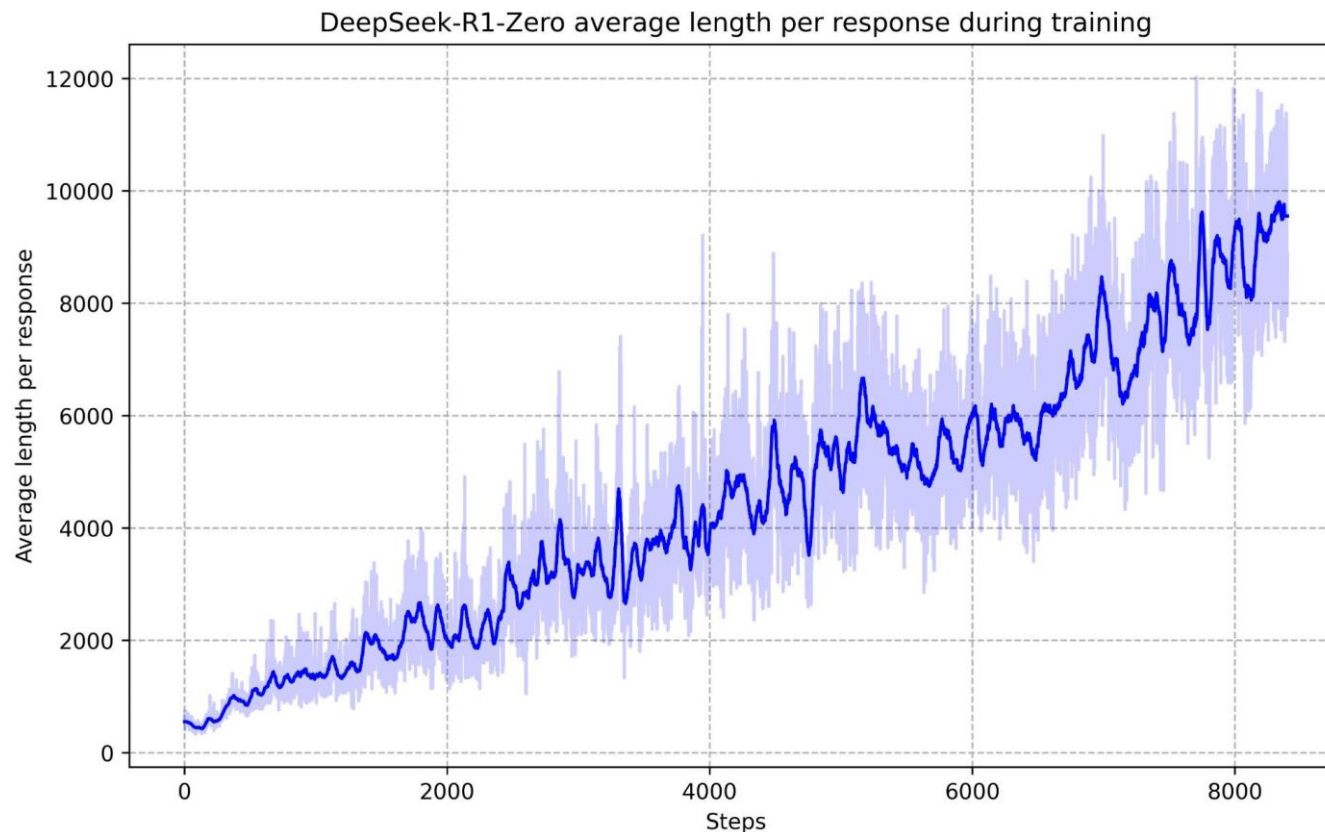
    entropy = -logps.sum() / labels_mask.sum() # scalar

    policy_loss = -logps * advantages[..., 1:] # [batch_size, seq_len-1]
    policy_loss = policy_loss * labels_mask # [batch_size, seq_len-1]

    loss = (policy_loss + KL_COEFFICIENT * kl_penalty).sum() / total_response_len # scalar
```

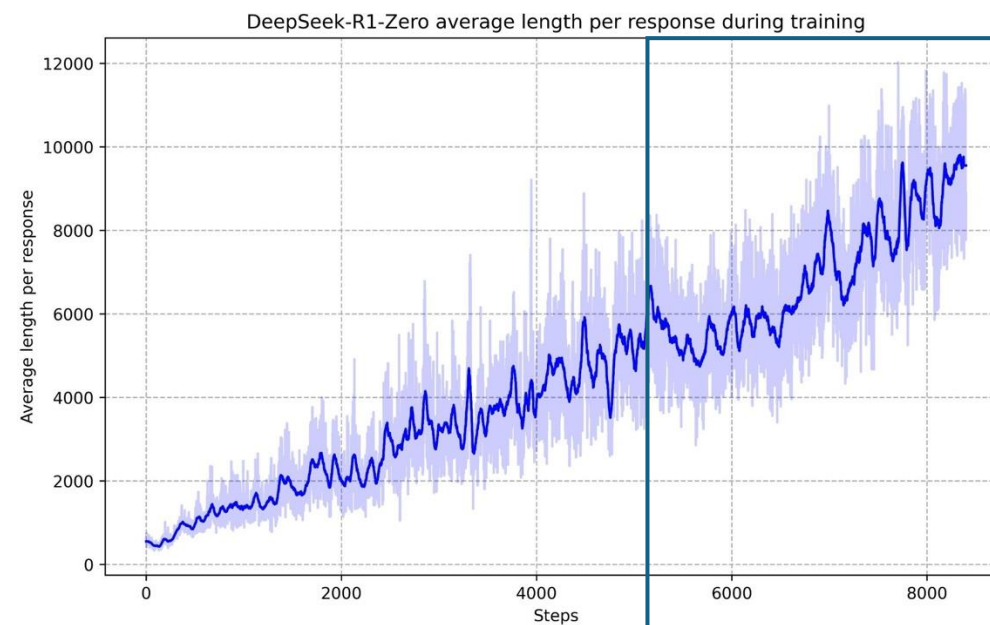
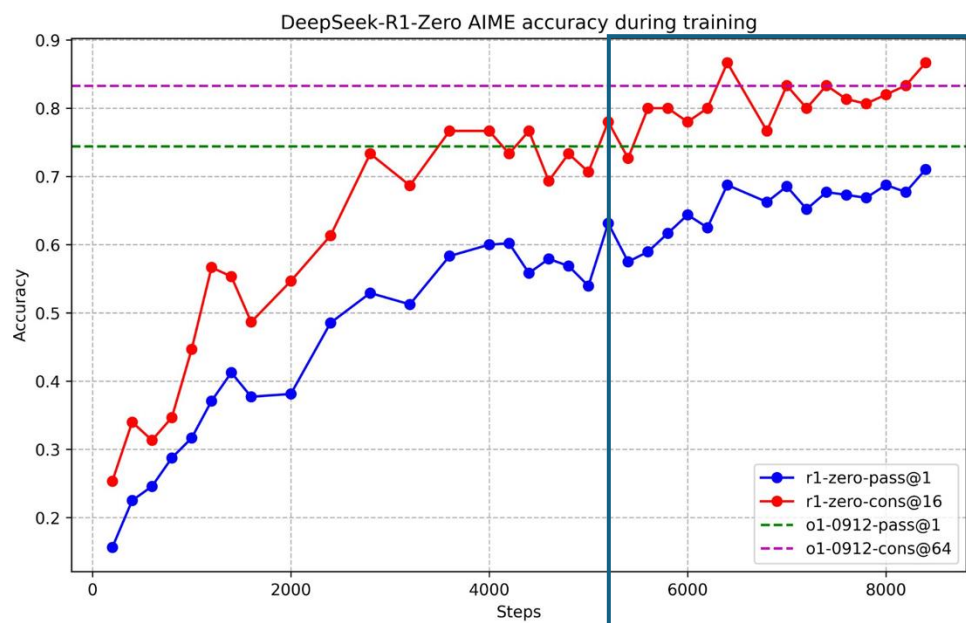
"Increasing output length" phenomenon

- Response length keeps on increasing with RL training



"Increasing output length" phenomenon

- Response length keeps on increasing with RL training



"Increasing output length" phenomenon

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

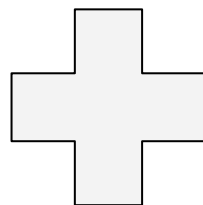
"Increasing output length" phenomenon

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \boxed{\frac{1}{|o_i|}} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \boxed{\frac{1}{|o_i|}} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

Small $|o|$



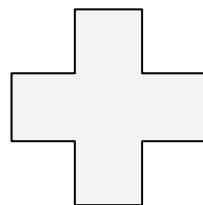
Big $|o|$



$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \boxed{\frac{1}{|o_i|}} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

Short output



Long output



$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

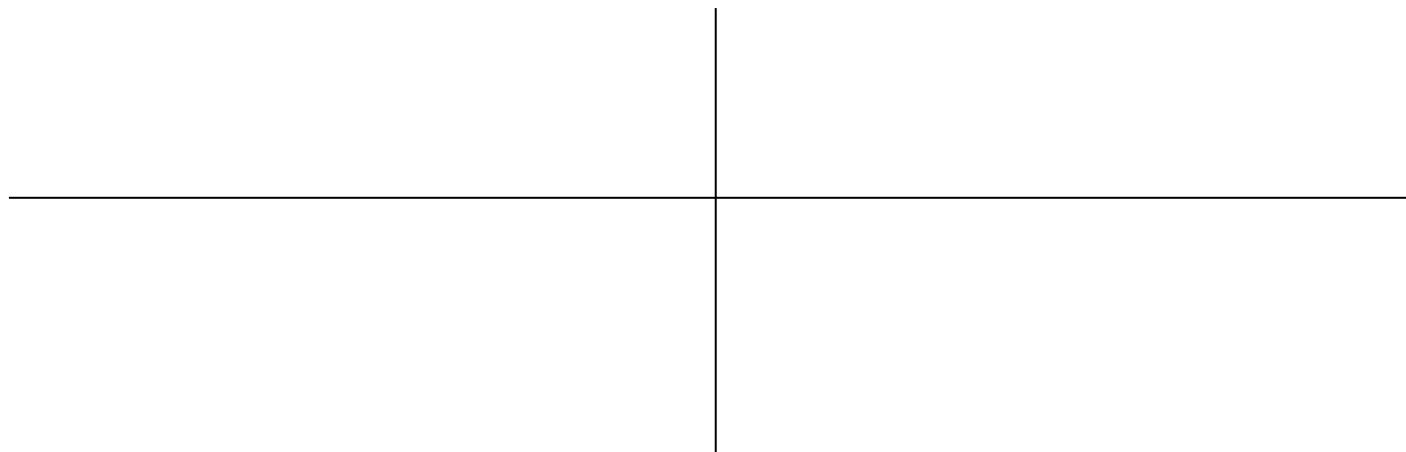
$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \boxed{\frac{1}{|o_i|}} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

$$A > 0$$

$$A < 0$$

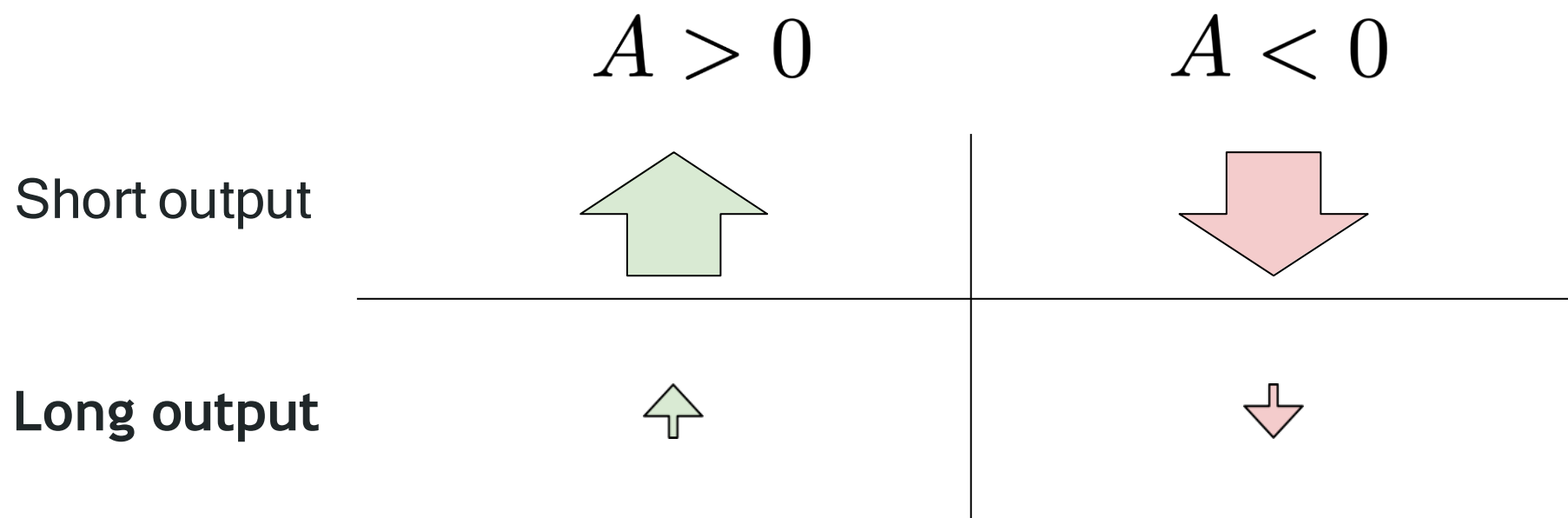
Short output

Long output



$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \boxed{\frac{1}{|o_i|}} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$



$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

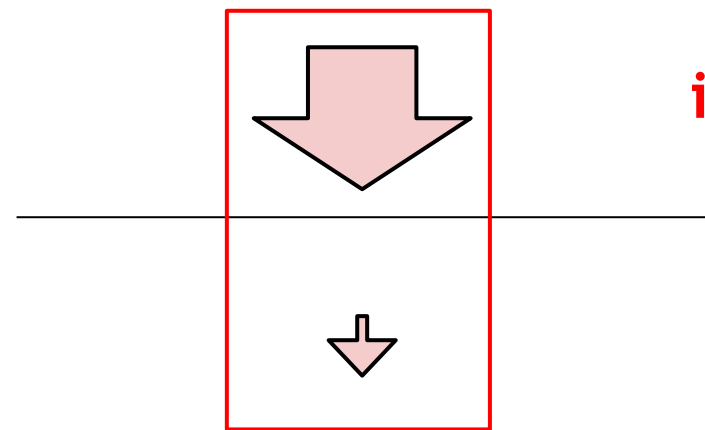
$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \boxed{\frac{1}{|o_i|}} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

$A > 0$

$A < 0$

Short output

Long output



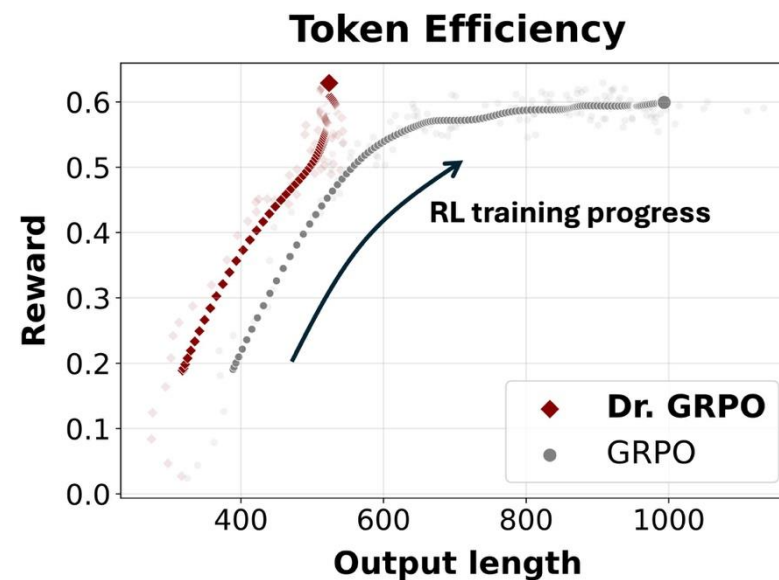
**Bad
incentive!**

Mitigating the length problem

- Equalize token-level contributions

- DAPO
$$\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|}$$

- Dr. GRPO
$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|}$$

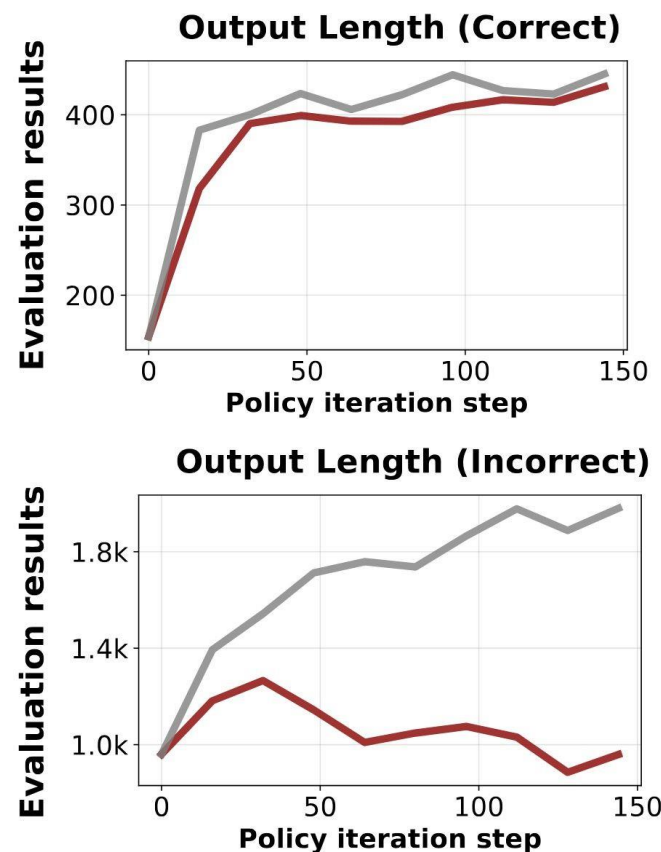


Mitigating the length problem

- Equalize token-level contributions

- DAPO
$$\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|}$$

- Dr. GRPO
$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|}$$



Other adjustments

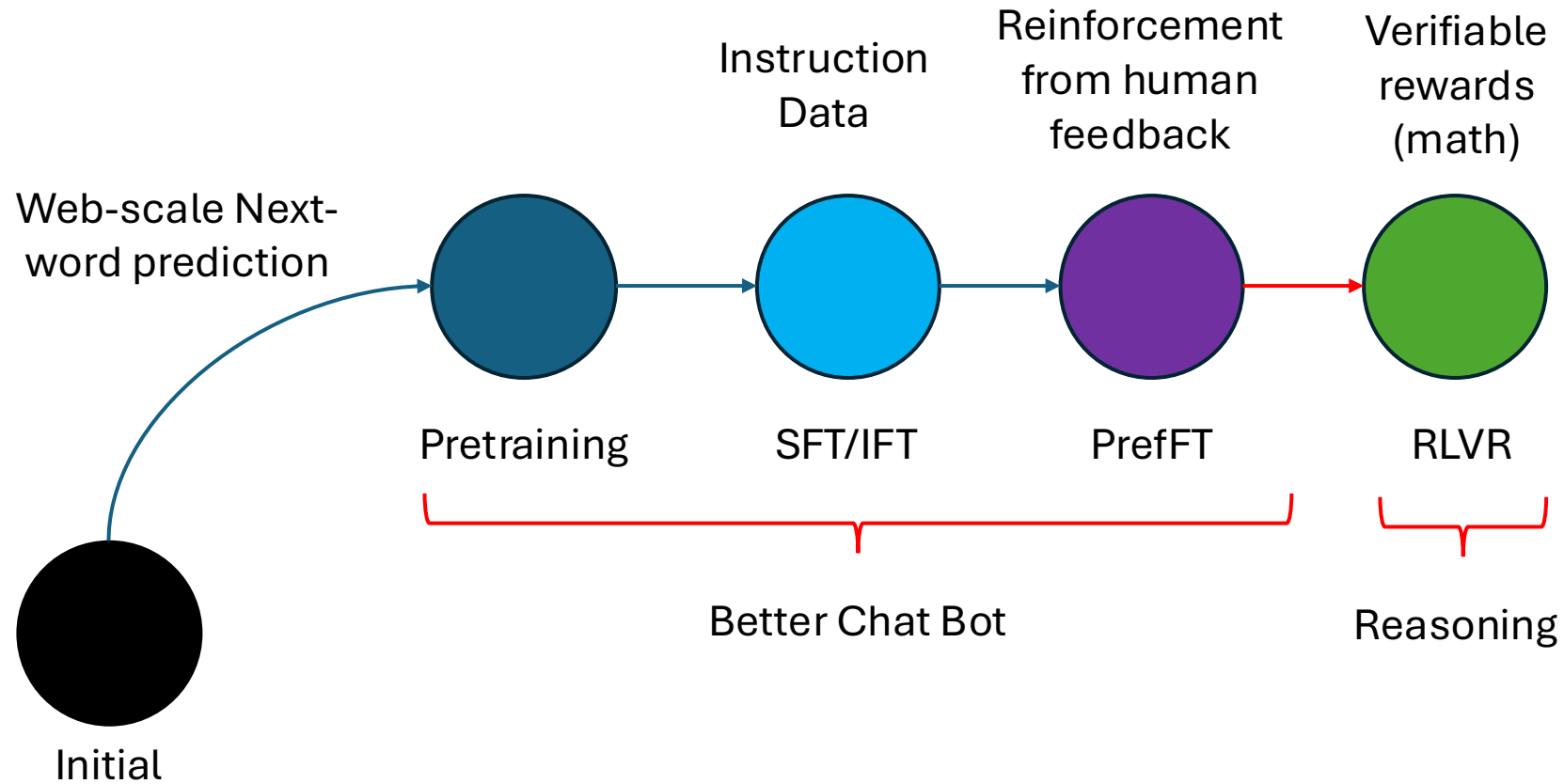
- Bias linked to level of difficulty

$$\hat{A}_{i,t} = \frac{R(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\{R(\mathbf{q}, \mathbf{o}_1), \dots, R(\mathbf{q}, \mathbf{o}_G)\})}{\text{std}(\{R(\mathbf{q}, \mathbf{o}_1), \dots, R(\mathbf{q}, \mathbf{o}_G)\})}$$

- Encourage diversity

$$\text{clip}\left(r_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon\right) \longrightarrow \text{clip}\left(r_{i,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}\right)$$

Path to O1/R1 models



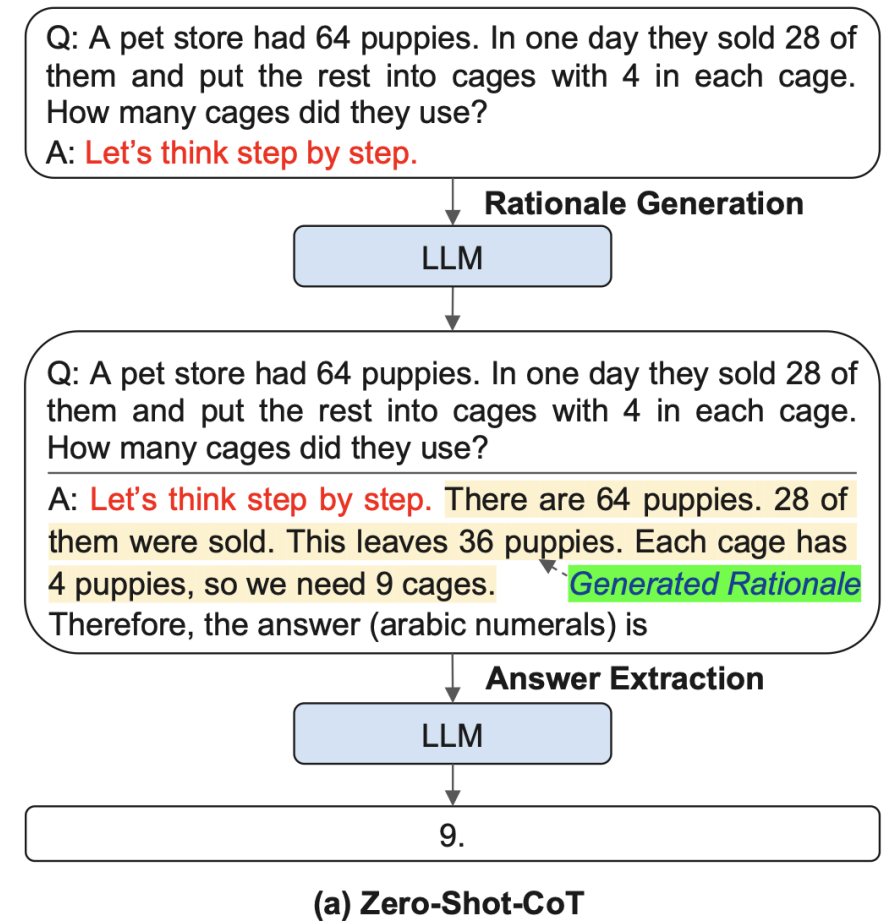
Reasoning

- the ability to solve novel problems by constructing and using intermediate abstractions or steps, rather than relying on direct retrieval
- Examples

Not reasoning	Reasoning
<i>"What is the course code of Stanford's Transformers & LLMs class?"</i>	<i>"The bear was born in 2020. How old is this bear now?"</i>

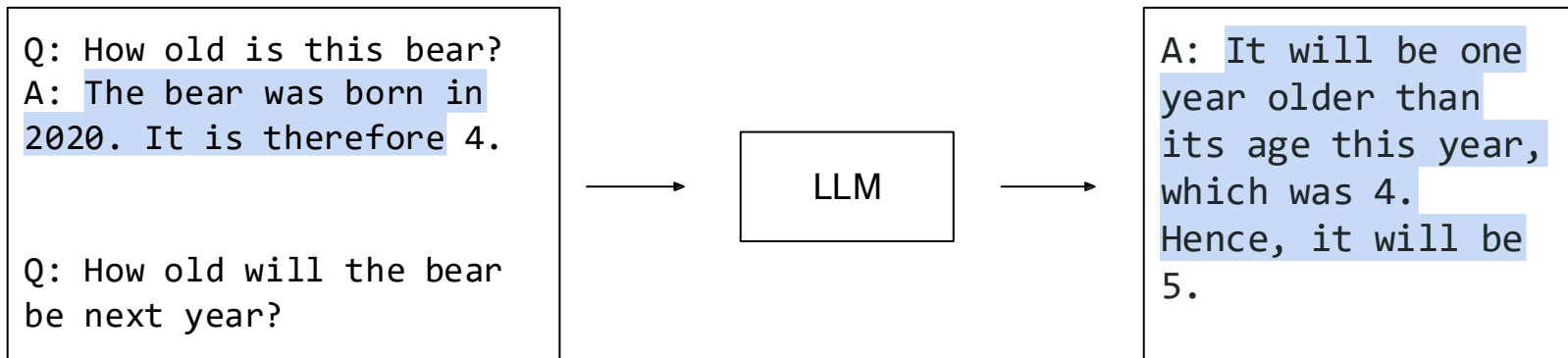
Reasoning models Inspired by Chain-of-Thought (CoT)

- “think step by step” prompt before answering a question [1]



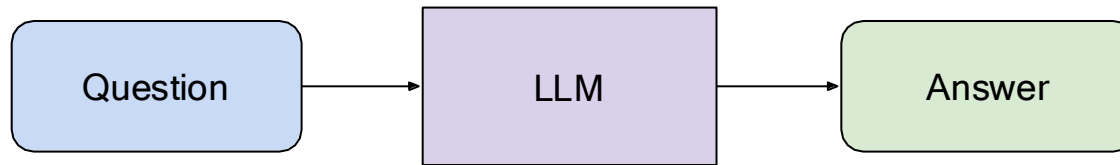
Example

- Explain its reasoning before answering (**Chain of Thought**)

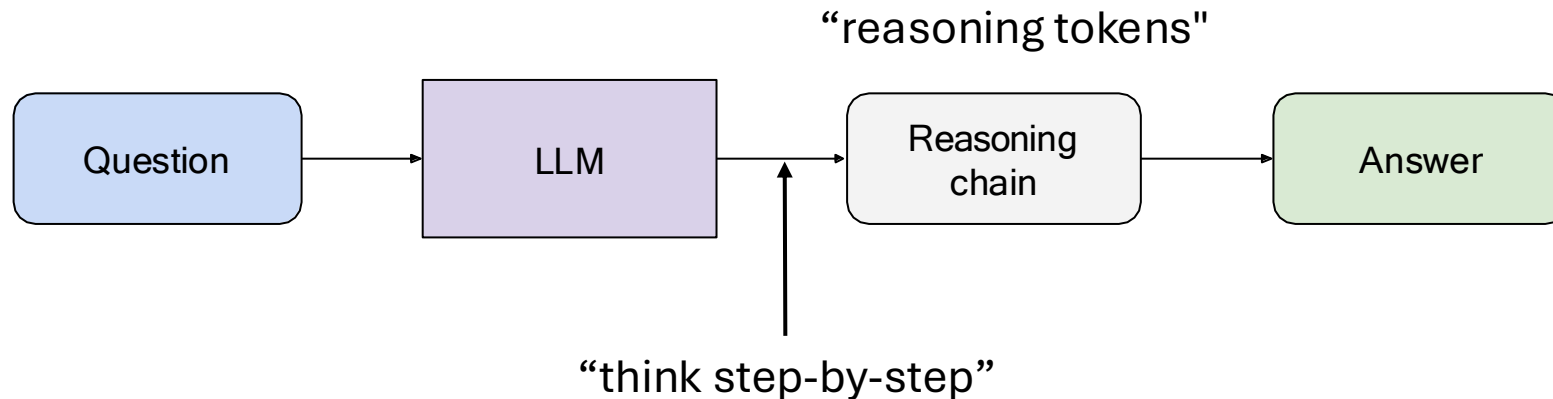


Enhancing model reasoning

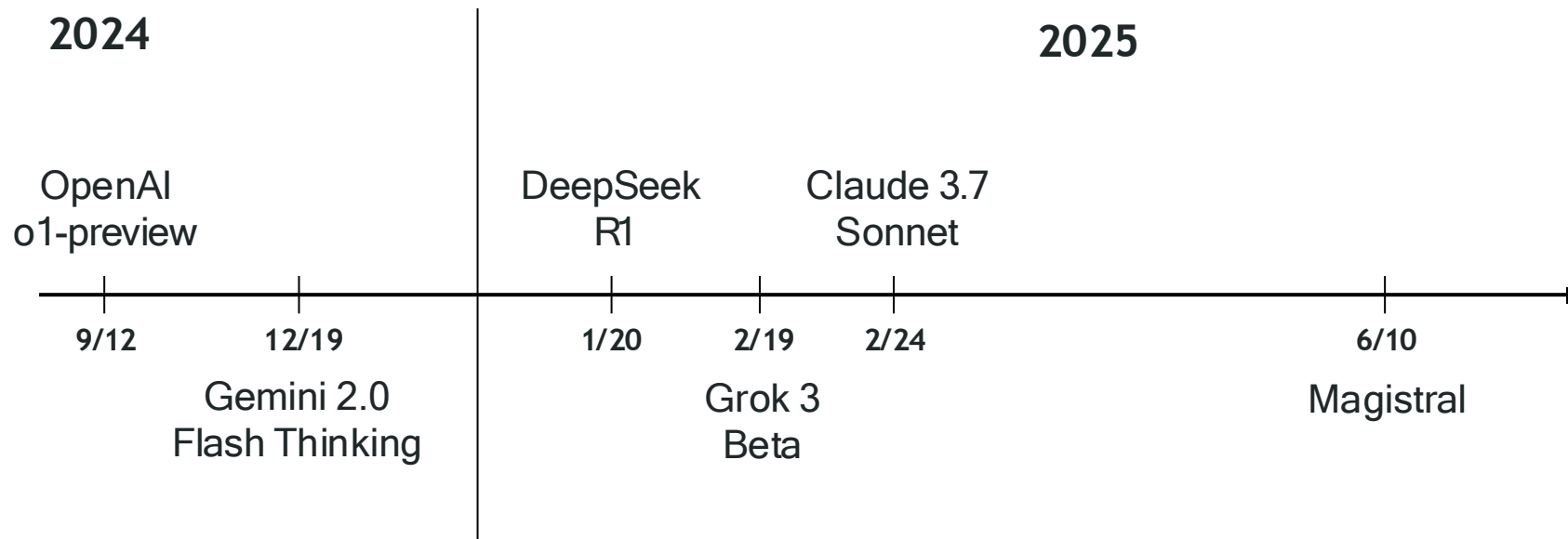
- Until now



- New paradigm



Reasoning models – the recent trend



Integrated in all state-of-the-art solutions

ChatGPT 5 Thinking

Share ...

Activity · 5s

×

My bear was born in 2020. How old is he now?

Thought for 5s >

As of today (November 3, 2025), he's either **5 years old** if his 2025 birthday has already passed, or **4 years old** if his birthday is later in November or December. If you share the exact date, I can give the precise age.



↓

Ask anything

+  Extended thinking



ChatGPT can make mistakes. Check important info.

Thinking

- Considering age calculation
The question is simple: the bear was born in 2020, and we need to calculate age as of November 3, 2025. If the bear's birthday has occurred before or on that day, they would be 5. If after November 3, they're still 4. So, either

✓ Thought for 5s

Done

Integrated in all state-of-the-art solutions

ChatGPT 5 Thinking ▾

Share ...

My bear was born in 2020. How old is he now?

Thought for 5s >

As of today (November 3, 2025), he's either **5 years old** if his 2025 birthday has already passed, or **4 years old** if his birthday is later in November or December. If you share the exact date, I can give the precise age.



↓

Ask anything

+  Extended thinking ▾



ChatGPT can make mistakes. Check important info.

Thinking tokens summary

Activity · 5s ×

Thinking

- Considering age calculation

The question is simple: the bear was born in 2020, and we need to calculate age as of November 3, 2025. If the bear's birthday has occurred before or on that day, they would be 5. If after November 3, they're still 4. So, either

Thought for 5s
Done

Notably,

- OpenAI

While reasoning tokens are not visible via the API, they still occupy space in the model's context window and are billed as output tokens.

- Anthropic

- You're charged for the full thinking tokens generated by the original request, not the summary tokens.
- The billed output token count will **not match** the count of tokens you see in the response.
- The first few lines of thinking output are more verbose, providing detailed reasoning that's particularly helpful for prompt engineering purposes.

Benchmarks

- Math*

id	problem	solution	answer
int64	string · lengths	string · lengths	string · lengths
60→62 10%	446→529 13.3%	634→1.54k 20%	3 100%
60	Every morning Aya goes for a 9-kilometer-long walk and stops at a coffee shop afterwards. When she walks at a constant speed of s kilometers per hour, the walk takes her 4 hours, including t minutes spent in the coffee shop. When she walks $s+2$ kilometers per hour, the walk takes her 2 hours and 24 minutes, including t minutes spent in the coffee shop. Suppose Aya walks at $s+\frac{1}{2}$ kilometers per hour. Find the number of minutes the walk takes her, including the t minutes spent in the coffee shop.	$\frac{9}{s} + t = 4$ in hours and $\frac{9}{s+2} + t = 2.4$ in hours. Subtracting the second equation from the first, we get, $\frac{9}{s} - \frac{9}{s+2} = 1.6$ Multiplying by $s(s+2)$, we get $9s+18-9s=18=1.6s^2 + 3.2s$ Multiplying by $5/2$ on both sides, we get $0 = 4s^2 + 8s - 45$ Factoring gives us $(2s-5)(2s+9) = 0$, of which the solution we want is $s=2.5$. Substituting this back to the first equation, we can find that $t = 0.4$ hours. Lastly, $s + \frac{1}{2} = 3$ kilometers per hour, so $\frac{9}{3} + 0.4 = 3.4$ hours, or $\boxed{204}$ minutes -Failure.net The amount of hours spent while walking on the first travel is $\frac{240-t}{6}$. Thus, we have the equation $(240-t)(s) = 540$, and by the same logic, the second equation yields $(144-t)(s+2) = 540$. We have $240s-st = 540$, and $288+144s-2t-st = 540$. We subtract the two equations to get $96s+2t-288 = 0$, so we have $48s+t = 144$, so $t = 144-48s$ and now we have $(96+48s)(s) = 540$. The numerator of	204

* - AIME, GSM8K

Benchmarks

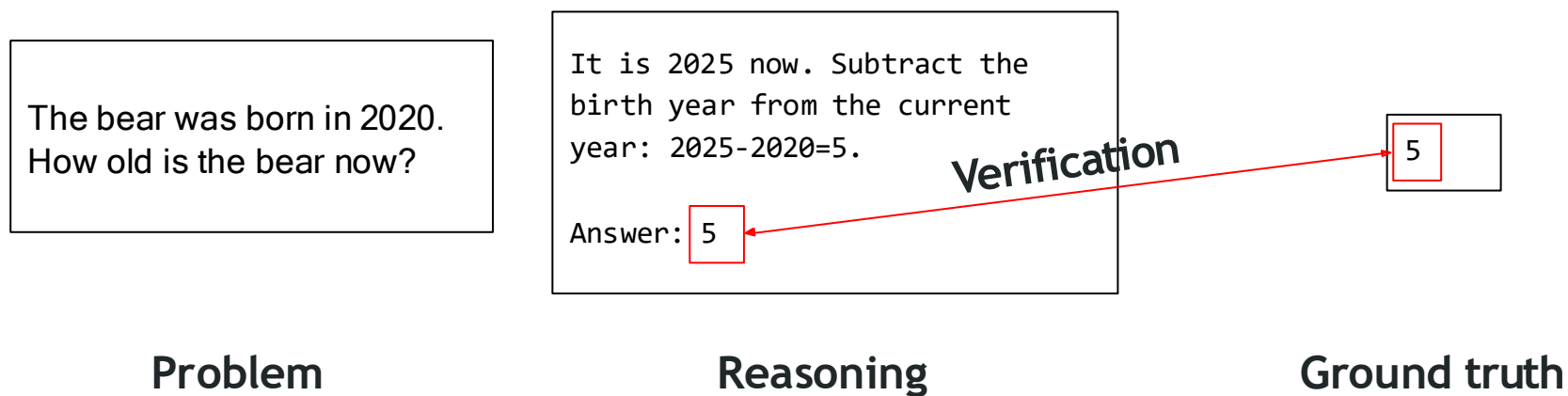
- Math*

id	problem	solution	answer
int64	string · lengths	string · lengths	string · lengths
60→62 10%	446→529 13.3%	634→1.54k 20%	3 100%
60	Every morning Aya goes for a 9-kilometer-long walk and stops at a coffee shop afterwards. When she walks at a constant speed of s kilometers per hour, the walk takes her 4 hours, including t minutes spent in the coffee shop. When she walks $s+2$ kilometers per hour, the walk takes her 2 hours and 24 minutes, including t minutes spent in the coffee shop. Suppose Aya walks at $s+\frac{1}{2}$ kilometers per hour. Find the number of minutes the walk takes her, including the t minutes spent in the coffee shop.	$\frac{9}{s} + t = 4$ in hours and $\frac{9}{s+2} + t = 2.4$ in hours. Subtracting the second equation from the first, we get, $\frac{9}{s} - \frac{9}{s+2} = 1.6$ Multiplying by $s(s+2)$, we get $9s+18-9s=18=1.6s^2 + 3.2s$ Multiplying by $5/2$ on both sides, we get $0 = 4s^2 + 8s - 45$ Factoring gives us $(2s-5)(2s+9) = 0$, of which the solution we want is $s=2.5$. Substituting this back to the first equation, we can find that $t = 0.4$ hours. Lastly, $s + \frac{1}{2} = 3$ kilometers per hour, so $\frac{9}{3} + 0.4 = 3.4$ hours, or $\boxed{204}$ minutes -Failure.net The amount of hours spent while walking on the first travel is $\frac{240-t}{6}$. Thus, we have the equation $(240-t)(s) = 540$, and by the same logic, the second equation yields $(144-t)(s+2) = 540$. We have $240s-st = 540$, and $288+144s-2t-st = 540$. We subtract the two equations to get $96s+2t-288 = 0$, so we have $48s+t = 144$, so $t = 144-48s$ and now we have $(96+48s)(s) = 540$. The numerator of	204

* - AIME, GSM8K

Evaluation metrics

- Verified metric that can be check correctness automatically, independent of human's subjective judgment.



Verified metrics are very common in code because of unit-tests

You have n teddy bears in a line. Each bear has a size.

Find the biggest bear that is smaller than the largest bear.

Problem

```
def second_biggest_bear(bears):  
    largest = max(bears)  
    return max(  
        b for b in bears  
        if b < largest  
    )
```

Solution

Test case #1 ✓

Test case #2 ✓

...

Test case #t ✓

Verification

Reasoning models with RL

- SFT data by hand impractical
- Reasoning chain is hard to write from scratch
- Also, no need to limit the model to human reasoning

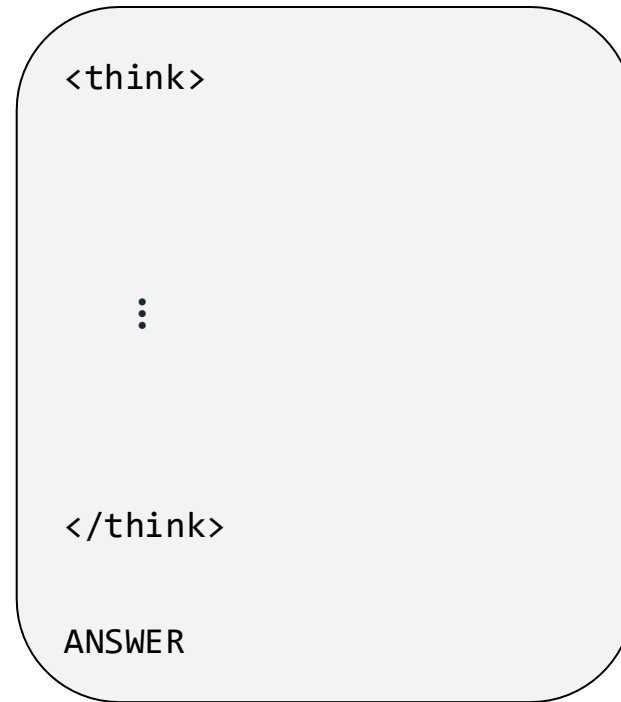
Instead of SFT, let's try RL

Reward for reasoning

- Usually there is no single correct reasoning path, only a correct outcome
- For reasoning we will need fast verified rewards
 - symbolic solvers
 - unit tests
- DeepSeek proposed elegant reward framework

R1:

Incentivize model to reason before answering



Template

Sample response

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

Reward #1: Verify CoT

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

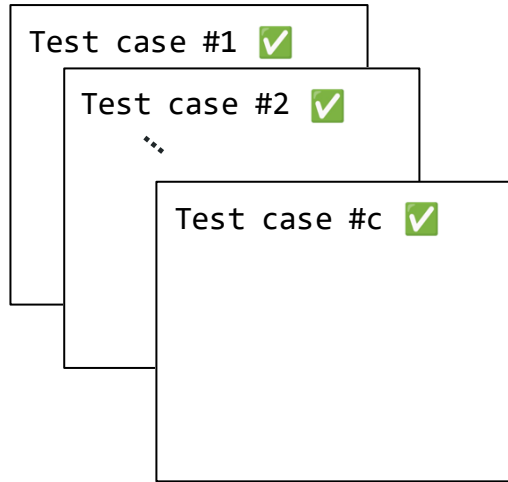
First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

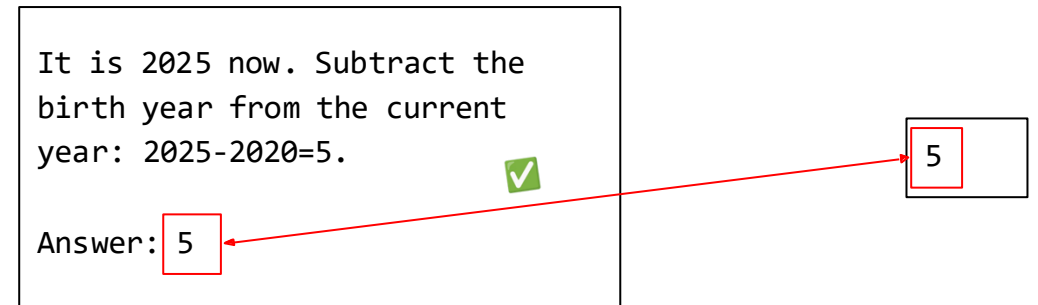
Next, I could square both sides again, treating the equation: ...

...

Reward #2: Verify correctness

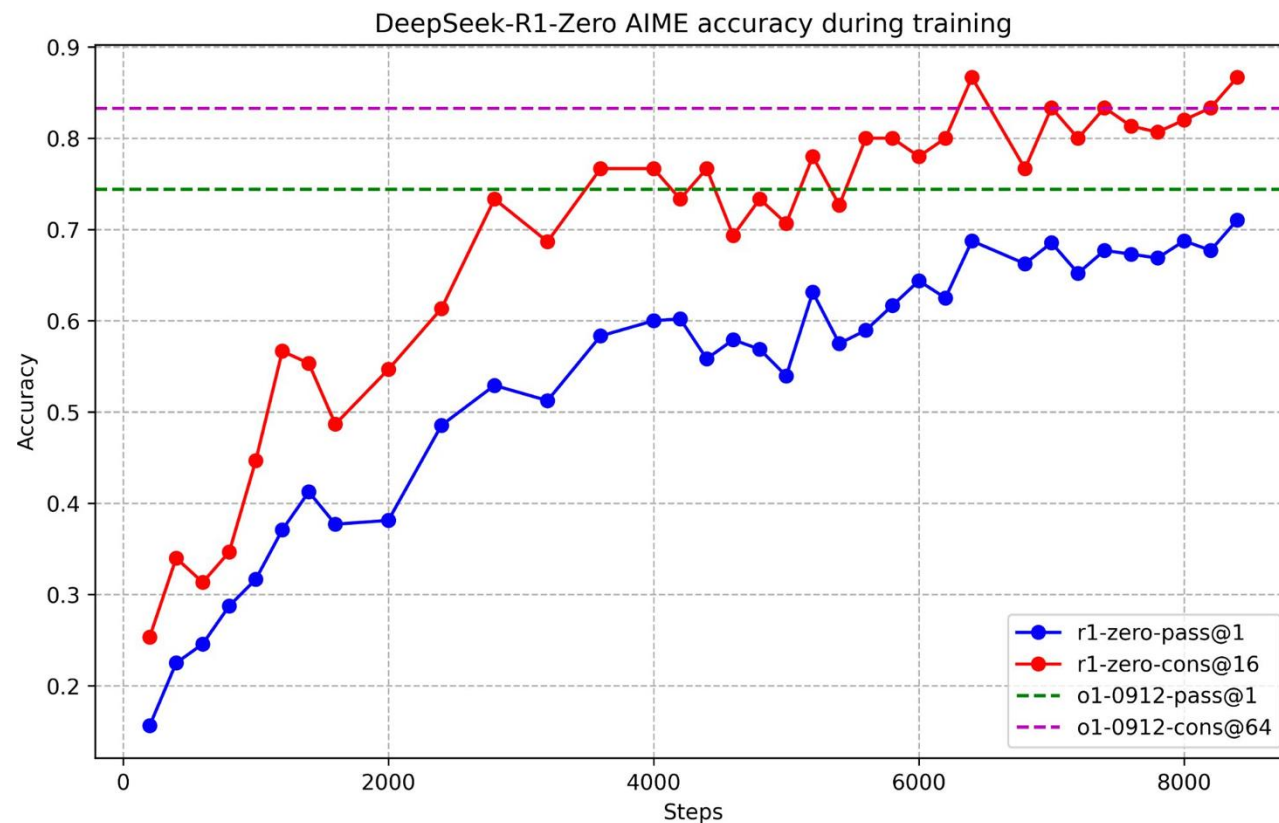
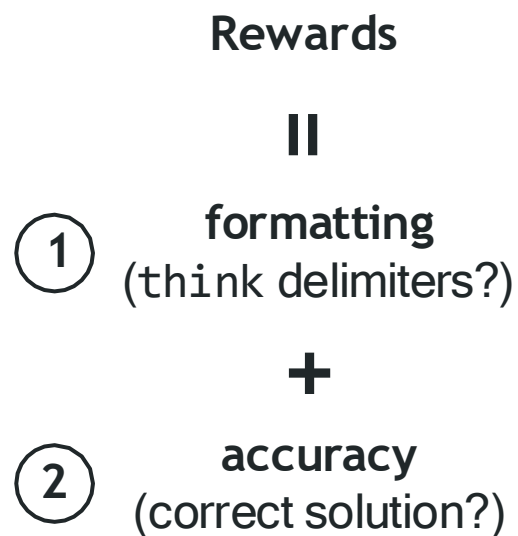


Code verification



Math verification

R1-Zero: RL with verifiable rewards



But what beat O1 was R1, The training recipe:

- Pretrain model with "traditional" techniques: **V3-Base**
 - MOE model
- Small-scale SFT with reasoning data
 - **Data:** long CoTs generated with R1-Zero and rewritten by humans
- GRPO with reasoning data
 - Same as R1-Zero training (Format + accuracy + language consistency)
- Large-scale SFT with Reasoning and non-Reasoning
 - 600k pairs reasoning math/code/logic (R1+rules/V3 judge)
 - 200k general SFT data (reuse V3)
- GRPO with reasoning and non-reasoning data
 - Math, code, Logic (Formatting + accuracy)
 - V3 RL data (helpfulness harmlessness)

Jaw-dropping Results

Benchmark (Metric)		Claude-3.5-Sonnet-1022	GPT-4o-0513	DeepSeek-V3	OpenAI-o1-mini	OpenAI-o1-1217	DeepSeek-R1
	Architecture	-	-	MoE	-	-	MoE
	# Activated Params	-	-	37B	-	-	37B
	# Total Params	-	-	671B	-	-	671B
English	MMLU (Pass@1)	88.3	87.2	88.5	85.2	91.8	90.8
	MMLU-Redux (EM)	88.9	88.0	89.1	86.7	-	92.9
	MMLU-Pro (EM)	78.0	72.6	75.9	80.3	-	84.0
	DROP (3-shot F1)	88.3	83.7	91.6	83.9	90.2	92.2
	IF-Eval (Prompt Strict)	86.5	84.3	86.1	84.8	-	83.3
	GPQA Diamond (Pass@1)	65.0	49.9	59.1	60.0	75.7	71.5
	SimpleQA (Correct)	28.4	38.2	24.9	7.0	47.0	30.1
	FRAMES (Acc.)	72.5	80.5	73.3	76.9	-	82.5
	AlpacaEval2.0 (LC-winrate)	52.0	51.1	70.0	57.8	-	87.6
	ArenaHard (GPT-4-1106)	85.2	80.4	85.5	92.0	-	92.3
Code	LiveCodeBench (Pass@1-COT)	38.9	32.9	36.2	53.8	63.4	65.9
	Codeforces (Percentile)	20.3	23.6	58.7	93.4	96.6	96.3
	Codeforces (Rating)	717	759	1134	1820	2061	2029
	SWE Verified (Resolved)	50.8	38.8	42.0	41.6	48.9	49.2
	Aider-Polyglot (Acc.)	45.3	16.0	49.6	32.9	61.7	53.3
Math	AIME 2024 (Pass@1)	16.0	9.3	39.2	63.6	79.2	79.8
	MATH-500 (Pass@1)	78.3	74.6	90.2	90.0	96.4	97.3
	CNMO 2024 (Pass@1)	13.1	10.8	43.2	67.6	-	78.8
Chinese	CLUEWSC (EM)	85.4	87.9	90.9	89.9	-	92.8
	C-Eval (EM)	76.7	76.0	86.5	68.9	-	91.8
	C-SimpleQA (Correct)	55.4	58.7	68.0	40.3	-	63.7

Jaw-dropping Results

Benchmark (Metric)		Claude-3.5-Sonnet-1022	GPT-4o-0513	DeepSeek-V3	OpenAI-o1-mini	OpenAI-o1-1217	DeepSeek-R1
English	Architecture	-	-	MoE	-	-	MoE
	# Activated Params	-	-	37B	-	-	37B
	# Total Params	-	-	671B	-	-	671B
	MMLU (Pass@1)	88.3	87.2	88.5	85.2	91.8	90.8
	MMLU-Redux (EM)	88.9	88.0	89.1	86.7	-	92.9
	MMLU-Pro (EM)	78.0	72.6	75.9	80.3	-	84.0
	DROP (3-shot F1)	88.3	83.7	91.6	83.9	90.2	92.2
	IF-Eval (Prompt Strict)	86.5	84.3	86.1	84.8	-	83.3
	GPQA Diamond (Pass@1)	65.0	49.9	59.1	60.0	75.7	71.5
	SimpleQA (Correct)	28.4	38.2	24.9	7.0	47.0	30.1
Code	FRAMES (Acc.)	72.5	80.5	73.3	76.9	-	82.5
	AlpacaEval2.0 (LC-winrate)	52.0	51.1	70.0	57.8	-	87.6
	ArenaHard (GPT-4-1106)	85.2	80.4	85.5	92.0	-	92.3
	LiveCodeBench (Pass@1-COT)	38.9	32.9	36.2	53.8	63.4	65.9
	Codeforces (Percentile)	20.3	23.6	58.7	93.4	96.6	96.3
Math	Codeforces (Rating)	717	759	1134	1820	2061	2029
	SWE Verified (Resolved)	50.8	38.8	42.0	41.6	48.9	49.2
	Aider-Polyglot (Acc.)	45.3	16.0	49.6	32.9	61.7	53.3
Chinese	AIME 2024 (Pass@1)	16.0	9.3	39.2	63.6	79.2	79.8
	MATH-500 (Pass@1)	78.3	74.6	90.2	90.0	96.4	97.3
	CNMO 2024 (Pass@1)	13.1	10.8	43.2	67.6	-	78.8
Chinese	CLUEWSC (EM)	85.4	87.9	90.9	89.9	-	92.8
	C-Eval (EM)	76.7	76.0	86.5	68.9	-	91.8
	C-SimpleQA (Correct)	55.4	58.7	68.0	40.3	-	63.7

Jaw-dropping Results

Benchmark (Metric)		Claude-3.5-Sonnet-1022	GPT-4o-0513	DeepSeek-V3	OpenAI-o1-mini	OpenAI-o1-1217	DeepSeek-R1
	Architecture	-	-	MoE	-	-	MoE
	# Activated Params	-	-	37B	-	-	37B
	# Total Params	-	-	671B	-	-	671B
English	MMLU (Pass@1)	88.3	87.2	88.5	85.2	91.8	90.8
	MMLU-Redux (EM)	88.9	88.0	89.1	86.7	-	92.9
	MMLU-Pro (EM)	78.0	72.6	75.9	80.3	-	84.0
	DROP (3-shot F1)	88.3	83.7	91.6	83.9	90.2	92.2
	IF-Eval (Prompt Strict)	86.5	84.3	86.1	84.8	-	83.3
	GPQA Diamond (Pass@1)	65.0	49.9	59.1	60.0	75.7	71.5
	SimpleQA (Correct)	28.4	38.2	24.9	7.0	47.0	30.1
	FRAMES (Acc.)	72.5	80.5	73.3	76.9	-	82.5
	AlpacaEval2.0 (LC-winrate)	52.0	51.1	70.0	57.8	-	87.6
	ArenaHard (GPT-4-1106)	85.2	80.4	85.5	92.0	-	92.3
Code	LiveCodeBench (Pass@1-COT)	38.9	32.9	36.2	53.8	63.4	65.9
	Codeforces (Percentile)	20.3	23.6	58.7	93.4	96.6	96.3
	Codeforces (Rating)	717	759	1134	1820	2061	2029
	SWE Verified (Resolved)	50.8	38.8	42.0	41.6	48.9	49.2
	Aider-Polyglot (Acc.)	45.3	16.0	49.6	32.9	61.7	53.3
Math	AIME 2024 (Pass@1)	16.0	9.3	39.2	63.6	79.2	79.8
	MATH-500 (Pass@1)	78.3	74.6	90.2	90.0	96.4	97.3
	CNMO 2024 (Pass@1)	13.1	10.8	43.2	67.6	-	78.8
Chinese	CLUEWSC (EM)	85.4	87.9	90.9	89.9	-	92.8
	C-Eval (EM)	76.7	76.0	86.5	68.9	-	91.8
	C-SimpleQA (Correct)	55.4	58.7	68.0	40.3	-	63.7

Thanks